

COMPSCI5103 Deep Learning — Closed-Book Exam Cheat Sheet (Memorise Everything Here)

ACTIVATION FUNCTIONS & DERIVATIVES (on crib sheet)

Sigmoid: $a(z) = 1/(1+\exp(-z))$ $da/dz = a(1-a)$

Saturates near 0,1 $\rightarrow$ vanishing gradient. Max derivative = 0.25 at $z=0$.

Tanh: $a(z) = (e^z - e^{-z})/(e^z + e^{-z})$ $da/dz = (1-a^2)$

Zero-centred. Still saturates. Used in GRU candidate state h_{tilde} .

ReLU: $a(z) = \max(0, z)$ $da/dz = 1$ if $z \geq 0$, else 0

No saturation for $z > 0$. Most common hidden activation. Derivative = gate (0 or 1).

Softmax: $a(z_i) = \exp(z_i) / \sum_j \exp(z_j)$

NOT the same as sigmoid. Use for multi-class. NEVER before CrossEntropyLoss.

OUTPUT SIZE FORMULA (on crib sheet)

$W_{\text{out}} = \text{floor}((W_{\text{in}} + 2P - F) / S) + 1$

W_{in} =input size F =filter/kernel size P =padding S =stride

Common cases to know cold:

Conv2d, stride=1, padding= $F//2 \rightarrow W_{\text{out}} = W_{\text{in}}$
(size preserved)

Conv2d, no pad, stride=1 $\rightarrow W_{\text{out}} = W_{\text{in}} - F + 1$

MaxPool(2x2, stride=2) $\rightarrow W_{\text{out}} = W_{\text{in}} / 2$
(halves)

MaxPool(2x2, stride=1) $\rightarrow W_{\text{out}} = W_{\text{in}} - 1$

Colour image [B, C, H, W]: B=batch C=channels H=height W=width

1D sequence [B, C, L]: B=batch C=channels L=length/timesteps

PARAMETER COUNTING — MEMORISE THESE

Linear(m, n) = $m*n + n$

$m*n$ weights + n biases (one bias per output neuron)

Conv2d(c_in, c_out, k) = $c_{\text{out}}*(c_{\text{in}}*k*k + 1)$

$c_{\text{in}}*k*k$ weights per filter + 1 bias per filter $\times c_{\text{out}}$ filters

Conv1d(c_in, c_out, k) = $c_{\text{out}}*(c_{\text{in}}*k + 1)$

Same formula but 1D kernel — k not $k*k$

Conv2d(c_in, c_out, kH, kW) = $c_{\text{out}}*(c_{\text{in}}*kH*kW + 1)$

NOTE: Sigmoid/ReLU/Softmax/Flatten have ZERO parameters

LOSS FUNCTIONS — WHICH TO USE WHEN

Binary classification (1 output, sigmoid):

`nn.BCELoss()` — expects sigmoid output in (0,1)

`nn.BCEWithLogitsLoss()` — expects raw logits (no sigmoid)

Multi-class (N outputs, no activation):

`nn.CrossEntropyLoss()` — includes LogSoftmax internally!

TRAP: Never add Softmax before CrossEntropyLoss — double-applied!

Regression (1 output, no activation / linear):

`nn.MSELoss()`

Target tensor for CrossEntropyLoss: shape [B], dtype torch.long, values 0..N-1

Target tensor for BCELoss: shape [B,1], dtype torch.float32, values 0.0 or 1.0

RNN FORWARD PASS — TEMPLATE (appears every year)

Equations:

$$h_t = \text{ReLU}(w \cdot h_{t-1} + u \cdot x_t)$$

$$y_t = c + v \cdot h_t$$

Step-by-step (given $u, v, w, c, h_{-1}=0$):

$$t=0: \text{inner}_0 = w \cdot h_{-1} + u \cdot x_0 = w \cdot 0 + u \cdot x_0 = u \cdot x_0$$

$$h_0 = \text{ReLU}(\text{inner}_0) = \max(0, u \cdot x_0)$$

$$y_0 = c + v \cdot h_0$$

$$t=1: \text{inner}_1 = w \cdot h_0 + u \cdot x_1$$

$$h_1 = \text{ReLU}(\text{inner}_1) = \max(0, \text{inner}_1)$$

$$y_1 = c + v \cdot h_1$$

ReLU check: if $\text{inner} < 0 \rightarrow h = 0 \rightarrow y = c$ (bias only)

Always state the ReLU input value and whether active or inactive!

BACKPROP & CHAIN RULE

Single neuron chain rule:

$$dL/dw = (dL/dy) \cdot (dy/dz) \cdot (dz/dw)$$

dy_0/dx_0 for vanilla RNN ($h_{-1}=0$):

$$\text{inner}_0 = u \cdot x_0$$

Case 1: $\text{inner}_0 \leq 0 \rightarrow \text{ReLU inactive} \rightarrow dy_0/dx_0 = 0$

Case 2: $\text{inner}_0 > 0 \rightarrow \text{ReLU active} \rightarrow dy_0/dx_0 = v \cdot u$

Sigmoid Jacobian (diagonal matrix):

$$dh/dz = \text{diag}(h_1(1-h_1), h_2(1-h_2))$$

Gradient through sigmoid network:

$$\text{grad}_h y = N^T = [w_1(2), w_2(2)]^T$$

$$\text{grad}_z y = \text{diag}(\text{sigmoid_derivs}) \cdot \text{grad}_h y$$

$$dy/dx_1 = w_1(2) \cdot h_1(1-h_1) \cdot w_{11}(1) + w_2(2) \cdot h_2(1-h_2) \cdot w_{12}(1)$$

Vanishing gradient: sigmoid max deriv=0.25. Deep network: $0.25^n \rightarrow 0$

OPTIMIZERS — SGD / MOMENTUM / ADAM

Plain SGD:

$$w = w - \text{lr} \cdot \text{grad}(L)$$

SGD + Momentum ($\text{beta}=0.9$ typical):

$$v_t = \text{beta} \cdot v_{t-1} + \text{grad}(L)$$

$$w = w - \text{lr} \cdot v_t$$

Helps with: narrow valleys/ravines — dampens oscillation, accelerates along valley

Weight Decay (L2 regularisation):

$$L_{\text{total}} = L_{\text{data}} + \text{lambda} \cdot \sum(w_i^2)$$

Penalises large weights $\rightarrow$ reduces overfitting $\rightarrow$ smaller, simpler model

PyTorch: `optim.SGD(..., weight_decay=0.001)`

Adam (momentum + adaptive lr):

$$m_t = b_1 \cdot m_{t-1} + (1-b_1) \cdot \text{grad} \quad (1\text{st moment})$$

$$v_t = b_2 \cdot v_{t-1} + (1-b_2) \cdot \text{grad}^2 \quad (2\text{nd moment})$$

$$w = w - \text{lr} \cdot m_{\text{hat}} / (\sqrt{v_{\text{hat}}} + \text{eps})$$

Innovation 1: Momentum — smooths direction. Innovation 2: Adaptive LR — scales per param

TRANSFORMER SELF-ATTENTION

Attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

Q =query (what am I looking for?) K =key (what do I have?) V =value (content)

$\sqrt{d_k}$ scaling prevents large dot products before softmax

Parameter count per head:

$$W_Q: d_e \times d_k \quad W_K: d_e \times d_k \quad W_V: d_e \times d_v$$

$$\text{Params per head} = d_e \cdot d_k + d_e \cdot d_k + d_e \cdot d_v$$

$$\text{Total (H heads)} = H \cdot (2 \cdot d_e \cdot d_k + d_e \cdot d_v) + H \cdot d_v \cdot d_o$$

Output projection W_O shape: $(H \cdot d_v) \times d_o$

Sequence length I NEVER appears in parameter count!

Example (2024 paper): $H=2, d_e=100, d_k=10, d_v=20, d_o=100$

$$\text{Per head: } 100 \cdot 10 + 100 \cdot 10 + 100 \cdot 20 = 1000 + 1000 + 2000 = 4000$$

$$2 \text{ heads: } 2 \cdot 4000 = 8000$$

$$\text{Output: } (2 \cdot 20) \cdot 100 = 4000$$

$$\text{Total: } 12000 \text{ params}$$

MANUAL CONVOLUTION — WORKED TEMPLATE

Kernel [-1,+1,0 / -1,+1,0 / -1,+1,0] on image (no pad, stride=1):

$$\text{output}(r,c) = \text{SUM over kernel: kernel}[i,j] * \text{image}[r+i, c+j]$$

Output rows = $\text{floor}((H+2P-kH)/S)+1$ Output cols = $\text{floor}((W+2P-kW)/S)+1$

Vertical edge kernel detects: left-dark right-bright transitions (positive response)

Horizontal edge kernel detects: top-dark bottom-bright transitions

Laplacian (0,-1,0/-1,4,-1/0,-1,0): detects blobs / point features

With padding=1, stride=2 (common exam pattern):

Add zero border of width 1 all around → image becomes (H+2) × (W+2)

Then apply formula: $W_{\text{out}} = \text{floor}((W+2 - kW)/2) + 1$

MaxPool(2x2, stride=2): take max of each non-overlapping 2x2 patch

MaxPool(2x2, stride=1): overlapping — output = $W_{\text{in}} - 1$ per dim

Channels NEVER change through MaxPool — only spatial dims shrink

GAN EQUATION + GRU GATES

GAN minimax:

$$\min_G \max_D V = E[\log D(x)] + E[\log(1-D(G(z)))]$$

G = Generator: noise $z \rightarrow$ fake image. D = Discriminator: image $\rightarrow$ prob real

$x \sim p_{\text{data}}(x)$: real sample. $D(x)$: prob real is real $\rightarrow$ D wants ≈ 1

$z \sim p_z(z)$: noise sample. $D(G(z))$: prob fake is real $\rightarrow$ D wants ≈ 0 , G wants ≈ 1

GRU Gates (2 gates, no separate cell state):

$r_t = \text{sigma}(W_r * [h_{(t-1)}, x_t])$ reset gate (θ =forget past)

$z_t = \text{sigma}(W_z * [h_{(t-1)}, x_t])$ update gate (θ =keep old h)

$h_{\text{tilde}} = \tanh(W * [r_t * h_{(t-1)}, x_t])$ candidate

$$h_t = (1 - z_t) * h_{(t-1)} + z_t * h_{\text{tilde}}$$

Sigmoid for gates: output in (0,1) = soft on/off switch

Tanh for candidate: output in (-1,1) = can increase OR decrease h

GRU advantage over vanilla RNN: update gate creates direct path

$\rightarrow$ gradient flows without shrinking $\rightarrow$ long-term memory preserved

Overfitting signs: train loss low, val loss rising. Fix with:

Dropout / More data / Weight decay / Early stopping / Reduce model size

PARAMETER COUNT — WORKED EXAMPLES

Linear(2,10) + Linear(10,10) + Linear(10,1):

$$fc1: 2 \cdot 10 + 10 = 30$$

$$fc2: 10 \cdot 10 + 10 = 110$$

$$fc3: 10 \cdot 1 + 1 = 11 \quad \text{TOTAL} = 151$$

Conv2d(1,8,3) + Conv2d(8,16,3) + Linear(16*8*8, 64) + Linear(64,3):

$$conv1: 8 \cdot (1 \cdot 3 \cdot 3 + 1) = 8 \cdot 10 = 80$$

$$conv2: 16 \cdot (8 \cdot 3 \cdot 3 + 1) = 16 \cdot 73 = 1168$$

$$fc1: (16 \cdot 64) \cdot 64 + 64 = 1024 \cdot 64 + 64 = 65600$$

$$fc2: 64 \cdot 3 + 3 = 195$$

$$\text{TOTAL} = 67043$$

Conv1d(20,5,15) + Conv1d(5,3,7) [2024 exam]:

$$conv1: 5 \cdot (20 \cdot 15 + 1) = 5 \cdot 301 = 1505$$

$$conv2: 3 \cdot (5 \cdot 7 + 1) = 3 \cdot 36 = 108$$

$$\text{Formula: } 100 \cdot v1 + 15 \cdot v2 + 8 \quad (v1=15, v2=7 \rightarrow 1613)$$

EXAM TRAPS & HIGH-VALUE FACTS

CrossEntropyLoss INCLUDES LogSoftmax — NEVER add Softmax before it

BCELoss needs sigmoid output. BCEWithLogitsLoss needs raw logits

Symmetric weight init (all same value) → identical gradients forever

Sequence length I NEVER affects Transformer parameter count

MaxPool does NOT change number of channels

Time: 20 marks = ~30 min = ~5 min per sub-part. Show ALL working.

Bias-variance: $\text{train} \downarrow \text{val} \uparrow$ = overfitting (high variance)

$\text{train} \uparrow \text{val} \uparrow$ = underfitting (high bias)

L2 vs L1: weight_decay = L2 (weights small but rarely zero)

L1 regularisation makes weights exactly zero (sparse)

Receptive field grows with depth:

2 layers of 3x3 conv → 5x5 receptive field in input

Conv(3x3) then MaxPool(2x2) → map back: pool pos × stride + kernel offset

Forward autodiff: P passes (one per param). Reverse (backprop): 2 passes total

This is why backprop is used — cost independent of number of parameters

COMPSCI5103 | For revision only — nothing permitted in closed-book exam except official crib sheet | Exam: Fri 9 May 2026 14:00 | 2 Questions × 20 marks = 40 marks | 60 min